STAR Archive Utility

Version 1.5.1

Technical Architecture, Operating Model and Validation

STAR archive format | Smart analysis | Global deduplication
Additive generations | Transaction safety | Verification

Document status

English technical documentation aligned with the reviewed SmartTAR - STAR 1.5.1 implementation. Product-specific paths, machine identifiers and user names are neutralized. Validation figures are observed test outcomes, not universal performance guarantees.

Updated: 16 October 2026

Contents

1. Executive overview
2. Core capabilities and compression modes
3. STAR archive architecture
4. Version 1.5.1 processing model
5. Smart analysis and preparation pipeline
6. Global file deduplication
7. Archive creation workflow
8. ADD generation workflow
9. Browse, extraction and restoration
10. Verification and transaction safety
11. Changes from version 1.4.0
12. Changes from version 1.5.0
13. Validation results
14. Operational guidance
15. Summary

Scope

This document explains the architecture and observed operating behavior of version 1.5.1. It is not a complete source-code API reference. Timings and storage savings depend on hardware, file count, data composition, cache state, antivirus activity and the selected compression profile.

Release alignment

Version 1.5.1 is a maintenance and hardening release over 1.5.0. The core STAR data model remains compatible: independent inner blocks, a browse catalog, direct full-file aliases, additive generations and one current canonical manifest. Version 1.5.1 formalizes block publication and the replaceable catalog/manifest tail and strengthens validation and compatibility paths.

1. Executive overview

STAR Archive Utility creates one portable .star file from folders or complete drive roots. The outer archive stores independently compressed inner TAR blocks, a compressed browse catalog and a JSON manifest that describes logical roots, physical blocks, direct aliases, history and current archive state.

Version 1.5.1 retains the integrity and additive-generation model of 1.4.0 and the sealed catalog pipeline introduced in 1.5.0. It adds implementation hardening around ZSTD block publication, canonical tail replacement, staging validation, worker execution and compatibility handling.

Capability	Version 1.5.1 behavior
Create	Build a new STAR archive from one folder or a complete drive root.
ADD	Append a logical generation while preserving existing physical blocks and deduplicating against all active stored content.
Browse / Extract	Browse from the compressed catalog and selectively restore a file, folder, root, ADD generation or the full archive.
Verify	Perform incremental structural checks or full streamed block and alias verification.
Delete	Remove logical roots through a newly published manifest without rewriting unaffected blocks.
Report	Produce human-readable operational and validation summaries.

Design objective

Avoid repeated source traversal, store only physically unique payloads, preserve direct logical aliases, publish archive metadata transactionally and keep extraction independent of the original workstation.

2. Core capabilities and compression modes

Mode	Behavior	Typical use
Balanced	General-purpose grouping with XZ9 for text/data where appropriate, ZSTD22 for disk-image-like data and STORE for media.	Mixed archives where ratio matters.
Smart	Extension and magic-byte classification plus sampled content analysis. Compressible unique data uses ZSTD22; media remains stored.	Large mixed datasets and content-aware grouping.
Solid	Places compressible unique data into ZSTD22-oriented blocks with reduced grouping overhead.	Homogeneous data where simplicity and ratio matter.
Store	Stores unique payloads without compression while preserving deduplication, cataloging and verification.	Already compressed content or speed-first archives.

Compression capability is proven by an actual round trip. The effective compressor thread count is capped at the smaller of eight or the logical processor count; unsupported multithread options are retried in single-thread compatibility mode.

3. STAR archive architecture

A STAR file is an outer TAR container with independently addressable data blocks and replaceable metadata at the tail.

Component	Purpose
blocks/...	Physically unique payloads stored using ZSTD22, XZ9 or STORE.
star-index.tar.zst	Compressed browse catalog used for fast navigation and selection.
manifest.json	Current authoritative logical model: roots, blocks, aliases, history, summaries and canonical tail metadata.
Alias map	Maps each duplicate logical path directly to a physically stored target and records its byte length.

Canonical tail

Version 1.5.1 uses canonical tail schema 2, identified as a replaceable catalog-manifest tail. The browse catalog is immediately followed by manifest.json, and manifest.json is the final logical outer entry. The manifest records the safe truncation offset used before a transactional ADD publication. No older catalog or manifest tails remain active after successful publication.

Direct aliases

Aliases always target a physical file, never another alias. This prevents alias chains and keeps extraction and verification deterministic across generations.

4. Version 1.5.1 processing model

The source is enumerated once into a sealed catalog. Safe relative paths, initial classification and profile totals are created during this pass. Later phases reuse the same records.

1. Enumerate the source once and build the sealed catalog.
2. Run Smart content analysis as a bounded embedded C# batch when selected.
3. Detect full-file duplicates and resolve direct physical targets.
4. Build unique-file and alias plans.
5. Create a destination-local .stw workroot only after source cataloging.
6. Stage and validate unique files.
7. Create one controlled inner block at a time.
8. Publish the compressed browse catalog and canonical manifest tail.
9. Run structural or full verification.

GUI worker model

Long-running GUI operations execute in a separate Windows PowerShell worker process. The parent GUI exchanges a compact configuration, status and result contract through a temporary per-operation workspace. The worker model isolates heavy archive work from the UI process; the workspace is removed during normal cleanup and does not alter the STAR format.

5. Smart analysis and preparation pipeline

Smart is the most demanding preparation mode because it analyzes content before physical block creation.

- File-extension classification establishes an initial group.
- Magic-byte detection recognizes common archive, media, executable and database signatures.
- For non-empty files, up to 1 MiB is sampled from the beginning and end.
- The analyzer calculates entropy, unique-byte count, null-byte count and a conservative text score.
- Analysis runs in embedded C# with bounded parallelism, avoiding per-byte PowerShell processing.

- The resulting group determines whether unique data enters ZSTD22 or STORE.

Preparation optimization

Cataloged paths, metadata and classifications are reused throughout planning. Per-item planning files are avoided where possible; expensive content reads and SHA-256 confirmations remain because those checks protect correctness.

6. Global file deduplication

Deduplication operates on complete files and applies to every non-empty file. Candidates are grouped by byte length. SHA-256 is calculated only when another file of the same size makes a duplicate possible. A confirmed duplicate becomes a direct logical alias to a physical target.

- Duplicate payloads are omitted from physical blocks.
- Alias targets must exist as physical archive content.
- Alias-to-alias chains are rejected.
- ADD generations deduplicate against existing active physical content as well as other files in the same incoming generation.
- An all-duplicate ADD generation may increase the archive only by catalog, manifest and TAR metadata overhead.

7. Archive creation workflow

10. Validate source and destination and exclude the transaction output from source discovery.
11. Build and seal the reusable source catalog.
12. Analyze content when Smart is selected.
13. Create the unique-file and alias plan.
14. Create the destination-local .stw workroot.
15. Stage only unique files and validate stage membership and sizes.
16. Create independent inner TAR blocks with the selected method.
17. For ZSTD blocks, validate and publish the exact frame length.
18. Build the browse catalog and manifest.
19. Publish the canonical tail and verify the result before final replacement.

Validated ZSTD frame publication

Version 1.5.1 captures the ZSTD output produced by the system tar process, parses the ZSTD frame structure and locates the actual frame end. Any trailing bytes accepted for removal must be zero padding and must remain within the expected output-padding limit. The block is published only after the trimmed size and frame boundaries pass validation. This prevents stdout padding from becoming part of the logical compressed block.

8. ADD generation workflow

ADD is transactional. Existing active data blocks are preserved. The incoming source is cataloged, analyzed and deduplicated against the archive-wide physical index. Only new unique payloads become new blocks.

20. Copy or prepare the transaction result without modifying the original archive.
21. Read the current manifest and canonical tail metadata.
22. Remove the replaceable catalog/manifest tail at the recorded safe offset.
23. Append only new physical blocks.
24. Merge existing and incoming roots, aliases, history and summaries.
25. Publish one new browse catalog and one canonical manifest.
26. Validate structure and the incremental generation.
27. Replace the original archive only after successful publication and validation.

The ADD logical root is mapped safely during extraction and never overwrites an unrelated user directory named ADD.

9. Browse, extraction and restoration

Browse first uses star-index.tar.zst, which avoids inspecting every physical block just to display the logical tree. A compatibility fallback can reconstruct required information from manifest and block data when an older or incomplete preview archive cannot use the fast catalog path.

Selective extraction

A producer/consumer stream bridge reads one compressed inner block from the outer STAR, calculates SHA-256 while streaming and feeds the selected bytes directly to the inner extractor. The compressed block is not written as an intermediate file. Restored aliases are materialized from their verified physical targets.

- Maintain safe relative paths and reject traversal outside the destination.
- Restore folder roots and drive roots through explicit virtual-root mapping.
- Map incremental ADD content to a distinct output root when required.
- Preserve compatibility handling for older preview layouts without weakening current validation.

10. Verification and transaction safety

Verification level	Coverage
Structural / incremental	Outer layout, canonical tail, manifest/catalog presence, new blocks, roots, alias targets and generation-specific invariants.
Full	Streams every active block, validates hashes and confirms every alias resolves to a physical file with the expected length.
Salvage support	Allows recoverable data to be inspected when metadata damage prevents the normal browse path; salvage is not a replacement for Verify.

The application does not publish a transaction result over the original archive until required validation succeeds. Cancellation and failure paths are designed to preserve the original or the last successfully published result.

11. Changes from version 1.4.0

Area	1.4.0 model	1.5.x model
Source discovery	Several helpers could enumerate or profile independently.	One sealed catalog is reused by later phases.
Planning	More per-item intermediate planning output.	Primarily in-memory planning with a compact final catalog and manifest.
Smart analysis	More orchestration overhead.	Bounded embedded C# batch analysis.
Compression	Earlier compressor selection and threading behavior.	ZSTD22/XZ9/STORE with round-trip capability testing and fallback.
ADD	Global model retained.	Dedup-aware staging and stronger transactional publication.
Diagnostics	Less centralized phase reporting.	Clearer status, validation and report summaries.

12. Changes from version 1.5.0

Version 1.5.1 is a maintenance and hardening release. It does not replace the STAR logical model introduced by 1.5.0.

- Added validated ZSTD frame publication that excludes accepted zero stdout padding from the final block.
- Formalized canonical tail schema 2 as a replaceable catalog-manifest tail with a recorded safe truncation offset.
- Strengthened stage validation, canonical-tail checks and incremental ADD publication checks.
- Clarified fast browse through the compressed catalog and compatibility fallback behavior.
- Hardened worker-process status/result handling and cleanup without changing archive semantics.
- Retained sealed catalog planning, global direct-alias deduplication, independent blocks and streamed selective extraction.

Compatibility note

The implementation continues to recognize supported earlier STAR/preview layouts through explicit compatibility paths. New archives and successful ADD publications use the current canonical model.

13. Validation results

The following values are observed results from anonymized practical tests retained from the reviewed 1.5.0 validation set. Version 1.5.1 changes publication and validation hardening; it does not recast these observations as new benchmarks.

13.1 Large mixed Smart dataset

Metric	Observed result
Logical source size	71.03 GB
Cataloged files	30,083
Unique files physically stored	13,612
Manifest aliases	16,471
Unique source bytes	51.92 GB
Alias bytes removed from physical blocks	19.11 GB
Final archive size	23.93 GB
Overall archive ratio	33.69%
Overall space saved	66.31%
ZSTD22 physical-data ratio	46.09%
Total operation time	04:56:58
Verified blocks	5 of 5
Verified aliases	16,471 of 16,471
Multithread fallback retries	0

Approximately 26.9% of logical byte volume was represented by aliases, while 54.8% of catalog entries were duplicate logical copies. ZSTD22 reduced the 51.92 GB unique payload to 23.93 GB.

13.2 Preparation observation

An observed Smart preparation phase completed in 32:05. Against a conservative 40-minute baseline, that represents approximately 19.8% less preparation time. The figure is a scenario-specific observation, not a universal guarantee.

13.3 Cross-archive ADD deduplication

A tested ADD generation containing only byte-identical payloads added a new logical root and direct aliases without new physical data blocks. The resulting archive passed incremental and full verification; growth consisted of catalog, manifest and TAR metadata overhead.

13.4 Validation limits

- The tests do not cover every Windows build, storage device, antivirus configuration or bsdtar implementation.
- Performance is sensitive to file count, random I/O, cache state, same-size candidates and source changes during analysis.
- Successful test results do not replace periodic Verify and independent backup copies.
- Fault injection, unusual reparse points, extreme path lengths and interrupted publication remain separate edge-case areas.

14. Operational guidance

- Run Verify after important ADD operations and before relying on a STAR archive as the only recovery copy.
- Keep enough free space for the transaction result, workroot, new unique blocks and a safety reserve.
- Do not manually edit manifest.json, the browse catalog or individual blocks inside a STAR archive.
- Keep TXT reports when diagnosing performance or integrity behavior.
- Use Smart when content-aware grouping and ratio matter; use Store when compression is unnecessary but deduplication and verification remain useful.
- For repeatable benchmarks, keep source, destination, cache condition, antivirus policy and background load consistent.
- Treat creation of the destination-local .stw workroot as a practical phase marker, not an exact first-compressed-byte timestamp.
- After cancellation or interruption, confirm that the original or resulting archive still passes Verify.

15. Summary

Version 1.5.1 is the hardened continuation of the 1.5.0 STAR architecture. It retains portable single-file storage, independent blocks, global full-file deduplication, additive generations, direct aliases, selective streaming and strong verification. The sealed catalog minimizes redundant preparation work, while ZSTD22/XZ9/STORE selection and proven threading capabilities provide practical compression behavior across Windows systems.

The 1.5.1 maintenance work strengthens exact ZSTD frame publication, formalizes the replaceable canonical catalog/manifest tail and improves staging, Browse, ADD and worker-process validation. These changes improve reliability without changing the core logical model or the observed validation claims inherited from 1.5.0.

Overall assessment

> **Version 1.5.1 is the preferred implementation for current use. The largest practical gains remain in catalog reuse, Smart preparation, dedup-aware staging, additive updates, streamed restoration and diagnostic clarity; the patch release adds publication and validation hardening around those paths.**